Async JavaScript for React

1. Synchronous JavaScript

Synchronous code normally runs one statement at a time. JavaScript waits for the current statement to finish before moving to the next.

```
console.log("A");
console.log("B");
console.log("C");
```

The output is A, B, C.

2. Asynchronous JavaScript

Asynchronous code starts an operation that may take time and allows other JavaScript to continue while it is waiting. This is useful for timers, file requests, APIs, and databases.

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 1000);

console.log("C");
```

The output is A, C, then B.

3. Why Async Code?

Web applications often wait for something outside the program, such as a server response. The exact time is unpredictable, so JavaScript needs a way to handle the result when it becomes available.

4. Promises

A Promise represents an operation that will finish in the future. It can be pending, fulfilled, or rejected.

```
const promise = new Promise((resolve) => {
  resolve("Success");
});

promise.then(result => {
  console.log(result);
});
```

then() runs when the Promise is successfully fulfilled.

5. fetch()

fetch() is commonly used to request a JSON file or data from an API. It returns a Promise.

```
fetch("students.json")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

response.json() also returns a Promise because the response must be converted into JavaScript data.

6. async and await

Adding async to a function allows await to be used inside it. await waits for a Promise inside that async function.

```
async function getData() {
  const response = await fetch("students.json");
  const data = await response.json();

  console.log(data);
}
```

async/await often makes asynchronous code easier to read because it looks like normal step-by-step code.

7. A Common Mistake

```
let data;

fetch("students.json")
  .then(response => response.json())
  .then(result => {
    data = result;
  });

console.log(data);
```

The console.log can run before fetch finishes, so data may still be undefined. Code that needs the result should run after the asynchronous operation finishes.

8. Error Handling

```
async function getData() {
  try {
    const response = await fetch("students.json");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Something went wrong", error);
  }
}
```

try/catch is commonly used with async/await to handle failed requests or other errors.

9. React: Fetch JSON with useEffect

React commonly loads external data inside useEffect(). Store the result in state so React can render it.

```
import { useEffect, useState } from "react";

function Students() {
  const [students, setStudents] = useState([]);

  useEffect(() => {
    async function loadStudents() {
      const response = await fetch("students.json");
      const data = await response.json();
      setStudents(data);
    }

    loadStudents();
  }, []);

  return (
    <div>
      {students.map(student => (
        <p key={student.id}>{student.name}</p>
      ))}
    </div>
  );
}
```

10. Why Not Make the Effect Callback async?

A common beginner pattern is to keep the useEffect callback itself synchronous and define an async function inside it. Then call that function.

```
useEffect(() => {
  async function loadData() {
    // asynchronous work
  }

  loadData();
}, []);
```

11. Loading State

Fetching takes time, so the interface can show a loading message while waiting.

```
const [students, setStudents] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  async function loadStudents() {
    const response = await fetch("students.json");
    const data = await response.json();
    setStudents(data);
    setLoading(false);
  }

  loadStudents();
}, []);

if (loading) return <p>Loading...</p>;
```

12. React Data Flow

A useful pattern to remember is: start with empty state → start the request → wait for the response → convert JSON → save it with setState → React renders the new data.

13. Key Points

• Synchronous code normally runs in order.
• Asynchronous operations can finish later.
• fetch() returns a Promise.
• then() handles a Promise result.
• async functions return Promises.
• await waits for a Promise inside an async function.
• Fetched data must be used after the request finishes.
• React commonly uses useEffect() for data-fetching side effects.
• useState() stores fetched data for rendering.